

Implementación *Sweet Crush* mediante manipulación de bits en C++

Este desafío consiste en diseñar e implementar el juego *Sweet Crush*, aplicando conceptos de operadores a nivel de bits (*bitwise*) y gestión de memoria dinámica, bajo restricciones estrictas de programación estructurada.

Objetivos

- Desarrollar la capacidad de solución de problemas en los estudiantes, enfrentándolos a problemáticas cercanas a situaciones reales.
- Evaluar si el estudiante ha adquirido las destrezas y conocimientos fundamentales de la programación en C++, tales como el uso de estructuras de control (secuenciales, iterativas y condicionales), manejo de tipos de datos, operaciones a nivel de bits, punteros, arreglos, memoria dinámica y funciones.

El objetivo principal de esta actividad es poner a prueba sus habilidades en el análisis de problemas y en el dominio del lenguaje C++. Si ha seguido un proceso disciplinado de aprendizaje a lo largo del semestre, esta es una excelente oportunidad para demostrarlo. Podrá proponer una solución efectiva y obtener un resultado satisfactorio. En caso contrario, esta actividad le permitirá identificar sus debilidades y tomar acciones correctivas con miras a prepararse mejor para enfrentar desafíos similares.

Se recomienda valorar adecuadamente la verdadera complejidad del problema planteado. No se rinda antes de intentarlo ni descarte posibles enfoques sin analizarlos. Descubrirá que, aunque al principio el reto pueda parecer difícil, ya ha desarrollado las herramientas necesarias para enfrentarlo con éxito. Si dedica el tiempo adecuado al análisis del problema, el proceso de codificación será mucho más fluido.

Esperamos que disfrute del desafío propuesto. Le sugerimos leer detenidamente todo el documento antes de comenzar, y asegurarse de entender completamente las instrucciones antes de abordar esta actividad evaluativa.

Fue revisado por los profesores Aníbal Guerra y Augusto Salazar.

Especificaciones

El propósito del desafío es diseñar e implementar una solución que permita administrar un tablero dinámico, realizar eliminaciones individuales de fichas, detectar combinaciones, modificar su estructura y gestionar las diferentes situaciones que se producen durante una partida. El tablero estará conformado por seis tipos diferentes de fichas . Cada ficha deberá ser identificable de manera inequívoca dentro del sistema.

Uno de los aspectos fundamentales del desafío consiste en que el tablero deberá almacenarse mediante una representación compacta a nivel de bits. No se permitirá utilizar un byte, un entero o cualquier otro tipo nativo completo para representar individualmente cada ficha.

A. Representación obligatoria de las fichas

Cada posición del tablero deberá representarse utilizando exactamente 3 bits. Con tres bits es posible representar ocho combinaciones diferentes: 000, 001, 010, 011, 100, 101, 110 y 111. De estas combinaciones deberán utilizarse seis para representar los seis tipos diferentes de fichas disponibles en el juego. Las dos combinaciones restantes podrán ser utilizadas por el estudiante para representar estados adicionales requeridos por su solución. Por ejemplo (ver Figura 1), una posición vacía, una posición especial u otro estado que considere necesario.

000 A Ficha A	001 B Ficha B	010 C Ficha C	011 D Ficha D
100 E Ficha E	101 F Ficha F	110 Estado libre	111 Estado especial

Figura 1. Ejemplo de representación de las fichas

La asignación concreta de los códigos será decisión del estudiante. Por ejemplo, una posible representación podría asociar diferentes fichas con diferentes patrones binarios, pero no existe una codificación obligatoria mientras cada posición utilice únicamente tres bits.

B. Representación obligatoria del tablero

El tablero completo deberá almacenarse como una secuencia continua de bits. Conceptualmente, si las fichas del tablero fueran: A B C D E F A B ..., internamente deberán almacenarse como una secuencia formada por los códigos de tres bits correspondientes a cada una: xxx xxx xxx xxx xxx xxx xxx ..., donde cada grupo xxx, representa exactamente una posición del tablero. El estudiante debe seleccionar adecuadamente los caracteres que mejoren la visualización en pantalla de las fichas y movimientos resultantes de la reacomodación del tablero.

Los bits correspondientes a una ficha deberán ubicarse inmediatamente después de los bits de la ficha anterior. No se permite reservar espacios adicionales con el propósito de alinear cada ficha a un byte.

Por tanto, el tablero no deberá interpretarse internamente como: 00000xxx 00000xxx 00000xxx ..., sino como una secuencia compacta: xxxxxxxx xxxxxxxx xxxxxxxx ..., en la que los bits de diferentes fichas pueden compartir un mismo byte. La Figura 2 ilustra la forma en la que se debe representar el tablero.

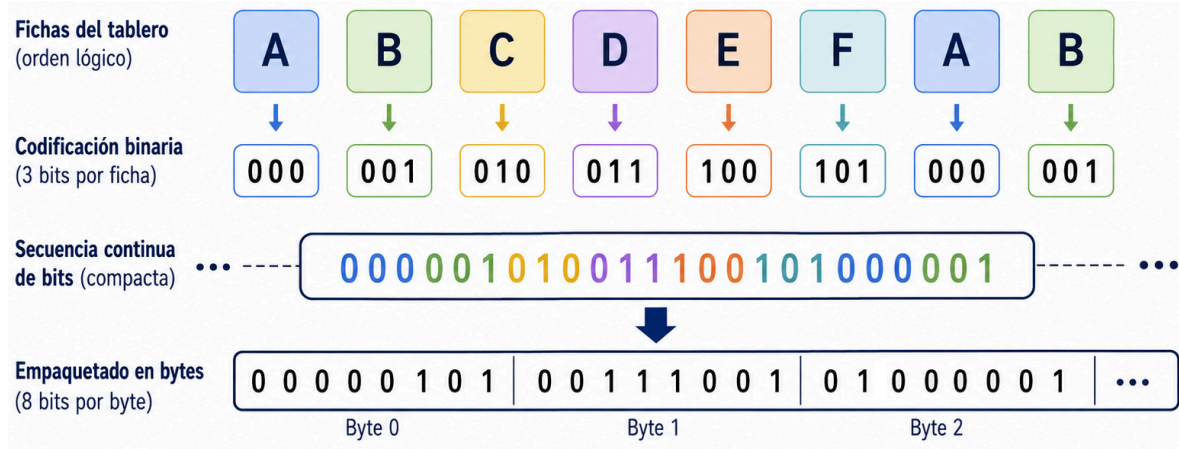


Figura 2. Representación del tablero

C. Distribución de las fichas entre bytes

Debido a que cada ficha ocupa tres bits y cada byte contiene ocho bits, las fronteras entre las fichas no necesariamente coincidirán con las fronteras entre los bytes. Como consecuencia, podrán presentarse dos situaciones: (i) Una ficha puede encontrarse completamente contenida dentro de un único byte, (ii) puede ocurrir que los tres bits correspondientes a una ficha se encuentren repartidos entre dos bytes consecutivos. La Figura 3 muestra tres situaciones que se pueden presentar.

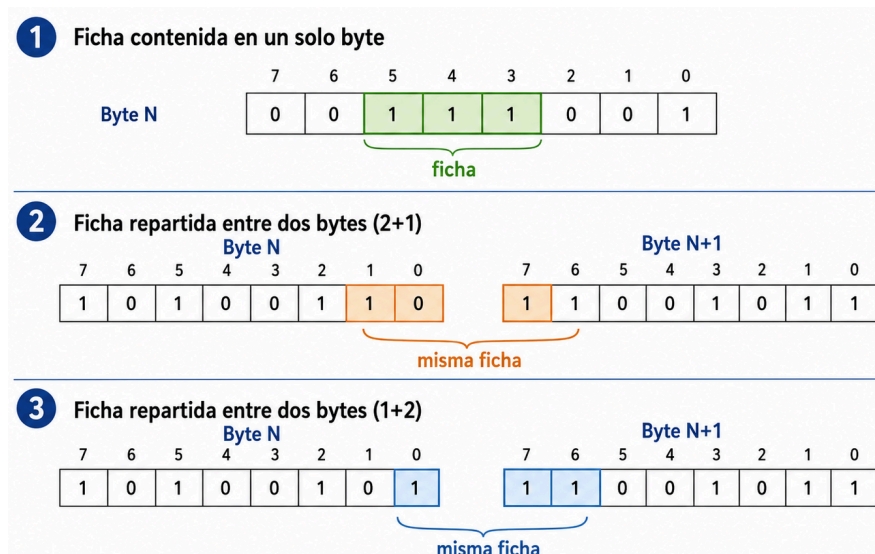


Figura 3. Distribución de una ficha dentro de los bytes

El programa deberá ser capaz de acceder y modificar correctamente una ficha independientemente de la forma en que sus tres bits estén distribuidos en la memoria.

D. Almacenamiento físico

El tablero deberá almacenarse utilizando memoria dinámica y bloques de bytes. La estructura física deberá estar basada en un puntero a memoria dinámica de tipo compatible con el almacenamiento de bytes, por ejemplo: `unsigned char *`. Sin embargo, cada elemento de dicho bloque de memoria no representa una ficha.

Cada byte forma parte únicamente del almacenamiento físico de la secuencia continua de bits del tablero. La relación entre: fila, columna, posición lógica, posición dentro de la secuencia de bits, byte correspondiente y bits correspondientes, deberá ser determinada por el estudiante durante la fase de análisis.

E. Tamaño del tablero en memoria

Para un tablero de: **F** filas y **C** columnas, existen: $F \times C$ posiciones. Como cada posición utiliza exactamente tres bits, la cantidad de bits requeridos será: $3 \times F \times C$. El estudiante deberá determinar la cantidad mínima de bytes necesaria para almacenar dicha información.

La memoria reservada deberá corresponder a las dimensiones actuales del tablero. No se permitirá reservar un arreglo suficientemente grande desde el inicio para evitar posteriores modificaciones del tamaño. La Figura 4 muestra la relación entre la posición de una ficha y su ubicación dentro de la memoria.

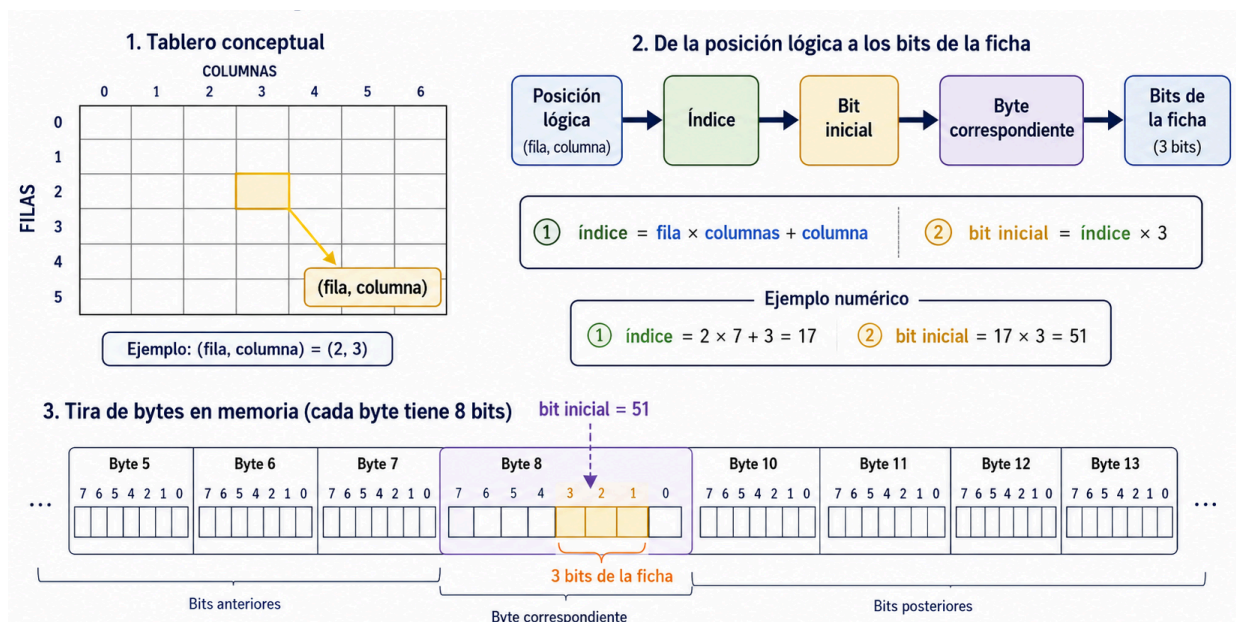


Figura 4. Relación entre la posición de una ficha y su almacenamiento dentro de la memoria asignada al tablero

F. Operaciones obligatorias a nivel de bits

Debido a la representación establecida, todas las operaciones necesarias para interpretar o modificar las fichas deberán realizarse mediante manipulación directa de bits.

El programa deberá utilizar de manera efectiva operadores como: &, |, ^, ~, << y >>, cuando sean necesarios para: localizar la información correspondiente a una ficha, extraer sus tres bits, construir su valor, modificar sus bits, entre otras operaciones.

El uso de operadores a nivel de bits deberá formar parte esencial del funcionamiento de la solución. No será válido desempaquetar permanentemente el tablero en una estructura auxiliar donde cada posición ocupe un byte o un entero y trabajar sobre dicha representación.

Requerimientos generales

El programa deberá permitir crear un tablero rectangular cuyas dimensiones iniciales serán indicadas al comenzar la ejecución. El sistema deberá permitir visualizar el estado actual del tablero de una manera comprensible para el jugador. Durante una partida, el jugador podrá seleccionar posiciones del tablero y eliminar las fichas en dicha ubicación.

Después de cada movimiento, el sistema deberá determinar si se han producido combinaciones válidas. Una combinación estará constituida por tres o más fichas iguales ubicadas consecutivamente en dirección horizontal o vertical. Cuando se detecte una combinación válida, las fichas involucradas deberán ser eliminadas del tablero. Una misma ficha podrá hacer parte simultáneamente de una combinación horizontal y de una combinación vertical. El sistema deberá garantizar que una ficha que participe simultáneamente en varias combinaciones sea procesada correctamente.

A. Reorganización del tablero

Después de una eliminación, las posiciones vacías generadas deberán ser gestionadas de forma que las fichas restantes se desplacen verticalmente para ocupar los espacios disponibles. Posteriormente, deberán incorporarse nuevas fichas para recuperar las dimensiones correspondientes del tablero.

La generación de nuevas fichas puede producir nuevas combinaciones sin que el jugador realice otra eliminación. El sistema deberá detectar automáticamente estas situaciones.

Se denominará cascada al proceso en el cual una eliminación provoca una reorganización del tablero que genera una o más combinaciones adicionales. El programa deberá continuar procesando las cascadas

hasta que el tablero alcance un estado en el cual no existan nuevas combinaciones generadas automáticamente.

B. Modificación dinámica de la estructura del tablero

Durante la ejecución podrán presentarse eventos que modifiquen las dimensiones del tablero.

El sistema deberá permitir:

- Agregar una nueva fila.
- Eliminar una fila existente.
- Agregar una nueva columna.
- Eliminar una columna existente.

Estas operaciones podrán realizarse sobre cualquier posición válida de la estructura y no estarán limitadas únicamente a los extremos del tablero. Por ejemplo, podrá agregarse o eliminarse una fila intermedia o una columna intermedia. Después de cualquier modificación estructural, el tablero deberá reorganizarse para continuar siendo completamente funcional. Las modificaciones deberán conservar correctamente todas las fichas que deban permanecer en el tablero.

Debido a que la representación se encuentra empaquetada en grupos continuos de tres bits, agregar o eliminar una fila o una columna puede modificar la ubicación física en memoria de una cantidad considerable de fichas. El estudiante deberá definir el comportamiento concreto de estas operaciones, siempre respetando la representación compacta del tablero y la gestión dinámica de memoria.

C. Modificación real de la memoria

Agregar o eliminar filas o columnas deberá modificar realmente las dimensiones de la estructura almacenada. Por tanto, cuando cambie la cantidad total de posiciones también deberá cambiar, cuando corresponda, la cantidad de memoria utilizada para representar el tablero.

No será válido:

- Mantener permanentemente las dimensiones máximas.
- Utilizar un tablero sobredimensionado.
- Reservar inicialmente memoria para todas las posibles expansiones futuras.
- Las tramas de bits válidos siempre deben iniciar en el bit menos significativo, y si existiesen bits “inválidos” o “vacíos”, estos deben agruparse a la izquierda de toda la trama.

En el caso de eliminar filas o columnas, la modificación de la memoria física sólo debe realizarse cuando la utilización de la memoria caiga debajo del 65 % respecto al tamaño actual del tablero.

D. Cascadas

Después de cualquier eliminación, modificación estructural o incorporación de nuevas fichas deberá verificarse si se producen nuevas combinaciones. Si aparecen nuevas combinaciones como consecuencia de una operación anterior, deberán ser procesadas automáticamente. Este procedimiento deberá repetirse mientras continúen apareciendo nuevas combinaciones.

El programa deberá llevar registro de las cascadas producidas durante la partida.

E. Estado del juego

El programa deberá mantener información suficiente para determinar, como mínimo:

- Dimensiones actuales del tablero.
- Cantidad de eliminaciones realizadas por el usuario.
- Cantidad de fichas eliminadas en total.
- Cantidad de combinaciones detectadas.
- Cantidad de cascadas producidas por la eliminación actual.
- Puntuación obtenida.

El criterio de puntuación podrá ser definido por el estudiante y deberá estar documentado.

Nota: La redacción final y las figuras utilizadas fueron realizadas con la asistencia de ChatGPT.

Criterios generales de evaluación

1. **Uso de operadores a nivel de bits (30%):** Uso eficiente de máscaras y desplazamientos para las características descritas en los apartados A al F de los requerimientos generales.
2. **Gestión eficiente de la memoria (35%):** Implementación y uso correcto de la memoria dinámica. Selección adecuada de las estructuras y tipos de datos. Balancear con la eficiencia en instrucciones.
3. **Lógica de juego (35%):** Funcionamiento correcto de:
 - Crear dinámicamente un tablero.
 - Generar las fichas iniciales de forma aleatoria, siguiendo una distribución uniforme.
 - Mostrar el tablero.
 - Eliminar la ficha indicada por el usuario.
 - Detectar combinaciones horizontales.
 - Detectar combinaciones verticales.
 - Resolver combinaciones simultáneas.
 - Reorganizar las fichas restantes.
 - Agregar una fila.
 - Eliminar una fila.
 - Agregar una columna.
 - Eliminar una columna.

- Gestión dinámica de la memoria requerida para almacenar el tablero.
- Visualización adecuada del tablero y sus fichas en pantalla (tanto en formato binario como en formato fichas).
- Cualquier otra requerida para satisfacer los requisitos establecidos.

Requisitos mínimos

A continuación, se describen los requisitos que se deben cumplir. El incumplimiento de cualquiera de ellos implica que su nota sea cero.

1. Genere un informe en donde se detalle el desarrollo del proyecto, explique entre otras cosas:
 - a. Análisis del problema y consideraciones para la alternativa de solución propuesta.
 - b. Esquema donde describa las tareas que usted definió en el desarrollo de los algoritmos.
 - c. Algoritmos implementados.
 - d. Problemas de desarrollo que afrontó.
 - e. Evolución de la solución y consideraciones para tener en cuenta en la implementación.
 - f. El informe no debe ser generado por alguna herramienta de IA.
2. La solución debe ser implementada en lenguaje C++ en el *framework* Qt. No incluir sintaxis de ANSI C.
3. La selección del tipo de variables y estructuras de datos debe hacerse responsablemente, pensando en uso eficiente de memoria. Las decisiones tomadas deben estar bien argumentadas.
4. La solución debe estar debidamente diseñada usando programación modular/multi-archivos (.h, .cpp).
5. No se pueden usar objetos tipo *string* como parte de la solución.
6. La implementación debe incluir el uso de punteros, arreglos y memoria dinámica.
7. No se puede usar *struct*, *class*, *template*, ni objetos definidos por el estudiante, así como estructuras dinámicas suministradas por bibliotecas, ni la STL.
8. Se debe crear un repositorio público en el cual se van a poder cargar todos los archivos relacionados a la solución planteada por usted (informe, código fuente y otros anexos).
9. Una vez cumplida la fecha de entrega no se podrá hacer modificación alguna al repositorio.
10. Se deben hacer *commits* de forma regular (repartidos durante el periodo de tiempo propuesto para las entregas) de tal forma que se evidencie la evolución de la propuesta de solución y su implementación.
11. Se debe adjuntar un enlace de *youtube* a un video que debe incluir lo siguiente:
 - a. Presentación de la solución planteada. Análisis realizado y explicación de la arquitectura del sistema (3 minutos máximo).
 - b. Demostración de funcionamiento del sistema. Explicar cómo funciona, abordando las siguientes situaciones:

- Eliminación de una ficha, que produzca una secuencia de al menos 3 fichas eliminadas (sentido horizontal y sentido vertical).
 - Eliminación de una fila y de una columna en posiciones intermedias del tablero.
 - Una eliminación que genere una cascada de al menos tres cascadas sucesivas.
 - Tiempo máximo para esta sección: 3 minutos .
- c. Explicación del código fuente. Tenga en cuenta que debe justificar la elección de las variables y estructuras de control usados. Por qué eligió uno u otro tipo de variable o estructura de control en cada caso particular y que ventaja ofrecen estos en comparación de otras que también podrían haber sido usados (5 minutos máximo).
 - d. La duración total del video no debe exceder 11 minutos ni ser inferior a 5 minutos.
 - e. Asegúrese que el video tenga buen sonido y que se puede visualizar con resolución suficiente para apreciar bien los componentes presentados.
 - f. El video no debe ser generado por alguna herramienta de IA.
 - g. En la elaboración del video deben participar todos los miembros del equipo. La colaboración de cada uno debe ser explícita (a través del audio descriptivo y/o video donde aparezcan).
11. El **plazo de entrega** se divide en dos momentos:
 - a. El día **11 de septiembre** (11:59 pm), para adjuntar la evidencia del proceso de análisis y diseño de la solución. (Informe preliminar: Contextualización, Análisis, Diseño).
 - b. El día **18 de septiembre** (11:59 pm), para adjuntar la evidencia del proceso de implementación.
 12. En Ude@ se deben adjuntar dos enlaces: uno al repositorio y otro al video, nada más.
 13. Para la evaluación del desafío se realizará una sustentación oral en un horario concertado con el profesor. La asistencia a la sustentación es obligatoria. Las sustentaciones que se realicen en modalidad virtual requerirán uso de cámara y el cumplimiento de condiciones de visualización por parte de los estudiantes.

Acotaciones finales

Ante la posibilidad de uso de herramientas de generación automática de soluciones se recuerda al estudiantado la responsabilidad que tiene de demostrar dominio analítico absoluto sobre la solución que entrega, así como control sobre las decisiones tomadas y el proceso de desarrollo ejecutado. Del mismo modo, se espera un conocimiento fluido de las instrucciones y del funcionamiento de su propio código. Todo lo anterior será verificado durante la sustentación, siendo esto un factor fundamental en el proceso de calificación.

No se podrá agendar sustentación en cualquiera de los siguientes casos:

- Si se omite la entrega, o se encuentra inaccesible algún enlace a: repositorio, informe, o video.
- Si la solución utiliza instrucciones de ANSI C.
- Si la solución no está debidamente diseñada usando programación modular/multi-archivos (.h, .cpp).
- Si la solución no hace uso de memoria dinámica en el core de la misma.
- Si la solución no hace uso de operadores a nivel de bits en el core de la misma.

- Si el core de la solución se implementa delegando tareas a código de autoría externa.
- Si se usan: *struct*, *class*, *template*, objetos definidos por el estudiante, estructuras dinámicas suministradas por librerías externas, entre ellas la STL.
- Si se incumple cualquier otra restricción establecida en este documento.